GAZİ UNIVERSITY

FACULTY OF ENGINEERING
DEPARTMENT OF INDUSTRIAL ENGINEERING

Lecturer : Dr Ercan Ezin

IE104-COMPUTER PROGRAMMING I

WEEK 9: LOOPS

Loops

- Loops are code blocks that allow certain tasks to be performed repeatedly within a program. Loops can be both infinite or controlled with a certain condition.
- There are four types of loop structures in the C# language:
  - **for**
  - **foreach**
  - **while**
  - **do while**

For Loop

```
for (expression; expression 2; expression3)
{
    statements;
}
```

- There are three expressions in a for loop separated by two semi-colons(;) .
- Some of these may be empty, but ";" sign must be used.
- **The first statement** is executed only once. It is generally used to define or initialize a loop variable.
- **The second statement** is the part where the loop condition is checked.
- The loop continues as long as the expression produces a “true” value.
- **The last statement** usually is the part where the value of the loop variable is changed.

For Loop

```
for(expr1;expr2;expr3)
{
   statements;
}
```

Usage:

- for(;;) is an infinite loop.
- for(int i=1;i<=100;i++)
- for(**int i=1,j=10;(i<=10)&(j>=1)**;**i++,j--)**

For Loop-Example

- Write a C# program that finds the sum of all numbers from 1 up to n which is entered from the keyboard.
- Expected screen is shown below:

C:\Users\AKIN\source\repos\IE_Gazi\IE_Gazi\bin\Debug\ne

```
Enter the value of n :10
Sum of value 1 to 10: 55
```

```csharp
static void Main(string[] args)
{
    // Example for Gazi IE
    int i, n, sum = 0;
    Console.Write("Enter the value of n :");

    n = Convert.ToInt32(Console.ReadLine());

    for (i = 1; i <= n; i++)
    {
        sum += i;
    }

    Console.WriteLine("Sum of value 1 to {0}: {1} ", n, sum);

    Console.ReadKey();
}
```

For Loop-Example

- Write a C# program that quits when user enters the word ''Quit''

Program.cs

IE_Gazi | IE_Gazi.Program

```
using System;

namespace IE_Gazi
{
    class Program
    {
        static void Main(string[] args)
        {
            // Example for Gazi IE
            string s;

            for (s = Console.ReadLine(); s != "Quit"; s = Console.ReadLine())
                Console.WriteLine(s);
            Console.ReadKey();
        }
    }
}
```

For Loop

- Write a C# program that displays the numbers from 0 to n according to the incremental value entered from keyboard.
- Expected screen is shown below:

```
C:\Users\AKIN\source\repos\IE_Gazi\IE_Gazi\bin\Debug\netcoreapp3.1'
Enter the value of n :7
Enter the incremental value:3
0
3
6
```

```
static void Main(string[] args)
{
    // Example for Gazi IE
    int i=0, n, inc;
    Console.Write("Enter the value of n :");
    n = Convert.ToInt32(Console.ReadLine());

    Console.Write("Enter the incremental value:");
    inc = Convert.ToInt32(Console.ReadLine());

    for (; i < n; )
    {
        Console.WriteLine("{0} ", i);
        i += inc;
    }
    Console.ReadKey();
}
```

For Loop

Example prime num program. Not the solution to the question on the left . You have to derive a solution from a similar way.

- Write a C# program that finds every available prime number numbers from first prime number to the ending number and prints the sum of found prime number to the screen.
- Expected screen is shown below:

Do this example and send me an email until tomorrow-
07 April 2026 23:59 deadline.

```
static void Main(string[] args)
{
    // Example for Gazi IE

    int k, t, sum, n1, n2;

    Console.Write("Starting number:");
    n1 = Convert.ToInt32(Console.ReadLine());
    Console.Write("Ending number:");
    n2 = Convert.ToInt32(Console.ReadLine());
    if (n1 <= n2)
    {
        for (k = n1; k <= n2; k++)
        {
            sum = 0;
            for (t = 1; t <= k; t++)
            {
                if (k % t == 0)
                    sum += t;
            }
            if (sum == k + 1)
                Console.WriteLine(k);
        }
    }
    else
        Console.WriteLine("Please enter a valid interval...");
    Console.ReadKey();
}
```

For Loop

- Alternative solution:
- Expected screen is shown below:

C:\Users\AKIN\source\repos\IE_Gazi\IE_Gazi\bin\Debug\ne

```
Starting number:5
Ending number:25
5
7
11
13
17
19
23
```

```csharp
static void Main(string[] args)
{
    // Example for Gazi IE
    int i, j, n1, n2, flag;

    Console.Write("Starting number:");
    n1 = Convert.ToInt32(Console.ReadLine());
    Console.Write("Ending number:");
    n2 = Convert.ToInt32(Console.ReadLine());
    if (n1 <= n2)
    {
        for (i = n1; i <= n2; i++)
        {
            flag = 0;
            for (j = 2; j < i; j++)
            {
                if (i % j == 0)
                { flag = 1;  break; }
            }
            if (flag==0)
            {
                Console.WriteLine(i);
            }
        }
    }
    else
        Console.WriteLine("Please enter a valid interval...");
    Console.ReadKey();
}
```

While loop

```
while (condition)
{
    statements
}
```

- The while loop continues to run as long as the specified condition is true. If you don't set a condition that makes while condition check result false or if you forget to enter a break statement, while loop will run infinitely that might cause your program to **freeze!**.

While and do-while Loop

```
do while
{
 statements
} while(condition);
```

In some cases, the loop may need to be run at least once. In this case «do while» can be used.

IMPORTANT NOTE: **If the condition check in for and while loops is not satisfied, the loop statement block is not executed at all.**

While and do-while Loop

C:\Users\AKIN\source\repos\IE_Gazi\IE_Gazi\bin\Debug\netcoreapp3.1\IE_Gazi.exe

5	10	15	20	25	30	35	40	45	50
55	60	65	70	75	80	85	90	95	100
105	110	115	120	125	130	135	140	145	150
155	160	165	170	175	180	185	190	195	200
205	210	215	220	225	230	235	240	245	250
255	260	265	270	275	280	285	290	295	300
305	310	315	320	325	330	335	340	345	350
355	360	365	370	375	380	385	390	395	400
405	410	415	420	425	430	435	440	445	450
455	460	465	470	475	480	485	490	495	500
505	510	515	520	525	530	535	540	545	550
555	560	565	570	575	580	585	590	595	600
605	610	615	620	625	630	635	640	645	650
655	660	665	670	675	680	685	690	695	700
705	710	715	720	725	730	735	740	745	750
755	760	765	770	775	780	785	790	795	800
805	810	815	820	825	830	835	840	845	850
855	860	865	870	875	880	885	890	895	900
905	910	915	920	925	930	935	940	945	950
955	960	965	970	975	980	985	990	995	1000

Write a program that outputs the values above. Make sure to cut the print statement when the number reaches 50 or its multipliers.

```
static void Main(string[] args)
{
    // Example for Gazi IE
    int i = 0;
    while (i < 1000)
    {
        i += 5;
        Console.Write("{0,5}", i);
        if (i % 50 == 0)
            Console.WriteLine();
    }


    Console.ReadKey();
}
```

While and do-while Loop

Write a program that does simple calculator operations. Make sure to prompt user a menu of options as shown in the image and do the operation when user selects an options. You can use switch statement or if else statement.

C:\Users\AKIN\source\repos\IE_Gazi\IE_Gazi\bin\Debug\netcoreapp3.1\IE_

```
Operations
========
1 - Addition
2 - Substraction
3 - Multiplication
4 - Division
0 - Quit

Choose your operation:9
Wrong Selection.. Try again please!

Operations
========
1 - Addition
2 - Substraction
3 - Multiplication
4 - Division
0 - Quit

Choose your operation:1
Addtion is selected.

Operations
========
1 - Addition
2 - Substraction
3 - Multiplication
4 - Division
0 - Quit

Choose your operation:0
You just wanted to quit.
```

```
static void Main(string[] args)
{
    // Example for Gazi IE
    int c;
   do
    {
   Console.WriteLine("Operations\n========");
   Console.WriteLine("1 - Addition");
   Console.WriteLine("2 - Substraction");
   Console.WriteLine("3 - Multiplication");
   Console.WriteLine("4 - Division");
   Console.WriteLine("0 - Quit\n");
   Console.Write("Choose your operation:");
   c = Convert.ToInt32(Console.ReadLine());
        switch (c)
        {
            case 1: Console.WriteLine("Addtion is selected.\n"); break;
            case 2: Console.WriteLine("Substraction is selected.\n"); break;
            case 3: Console.WriteLine("Multiplication is selected.\n"); break;
            case 4: Console.WriteLine("Division is selected.\n"); break;
            case 0: Console.WriteLine("You just wanted to quit.\n"); break;
            default:
                Console.WriteLine("Wrong Selection.. Try again please!\n"); break;
        }
        } while (c != 0) ;

        Console.ReadKey();
}
```

While and do-while Loop

Alternative solution

```csharp
using System;

class Program
{
    static void Main()
    {
        int choice;
        do
        {
            Console.WriteLine("===== SIMPLE CALCULATOR =====");
            Console.WriteLine("1. Addition");
            Console.WriteLine("2. Subtraction");
            Console.WriteLine("3. Multiplication");
            Console.WriteLine("4. Division");
            Console.WriteLine("5. Exit");
            Console.Write("Enter your choice: ");
            choice = Convert.ToInt32(Console.ReadLine());
            if (choice >= 1 && choice <= 4)
            {
                Console.Write("Enter first number: ");
                double num1 = Convert.ToDouble(Console.ReadLine());
                Console.Write("Enter second number: ");
                double num2 = Convert.ToDouble(Console.ReadLine());
                switch (choice)
                {
                    case 1:
                        Console.WriteLine("Result = " + (num1 + num2));
                        break;
                    case 2:
                        Console.WriteLine("Result = " + (num1 - num2));
                        break;
                    case 3:
                        Console.WriteLine("Result = " + (num1 * num2));
                        break;
                    case 4:
                        if (num2 != 0)
                            Console.WriteLine("Result = " + (num1 / num2));
                        else
                            Console.WriteLine("Error: Division by zero is not allowed.");
                        break;
                }
            }
            else if (choice == 5)
            {
                Console.WriteLine("Exiting program...");
            }
            else
            {
                Console.WriteLine("Invalid choice. Please select from 1 to 5.");
            }
            Console.WriteLine();
        } while (choice != 5);
    }
}
```

Foreach Loop

- Different than the other loop structures.
- Provides step-by-step navigation through collection-based objects.
- This loop provides an access to individual elements one by one, but the accessed elements are read-only (not changeable).

Foreach Loop

```
using System;

class Program
{
    static void Main()
    {
        // For loop example
        Console.WriteLine("For loop example:");
        for (int number = 1; number <= 5; number++)
        {
            Console.WriteLine("Number: " + number);
        }

        Console.WriteLine();

        // Foreach loop example
        Console.WriteLine("Foreach loop example:");
        string[] items = { "Pen", "Book", "Bag", "Pencil" };

        foreach (string item in items)
        {
            Console.WriteLine("Item: " + item);
        }
    }
}
```

Output

```
For loop example:
Number: 1
Number: 2
Number: 3
Number: 4
Number: 5

Foreach loop example:
Item: Pen
Item: Book
Item: Bag
Item: Pencil
```

Jump Statements

- Break: A running loop can be stopped by using the word “**break**.” After that, program execution continues with the lines that come after the loop. The word **break** can only be used in loops and switch statements.

- Continue: Allows the loop to move on to the next **iteration**.

- Goto: It is used to jump to another part of the program marked by a label. It is not very common in object-oriented programming and should be avoided as much as possible, except in a switch statement.

- Return: It is used to end a method at any time. Once the method ends, program flow continues from the function which the method was called.

Jump Statements-Example

```
using System;

class Program
{
    static void Main()
    {
        for (int number = 1; number <= 5; number++)
        {
            if (number == 2)
                continue;

            if (number == 4)
                break;

            Console.WriteLine("Number: " + number);
        }

        goto End;

    End:
        Console.WriteLine("Goto statement used.");
        ShowMessage();
    }

    static void ShowMessage()
    {
        Console.WriteLine("Return statement used.");
        return;
    }
}
```

Jump Statements-Exercise

- Write a C# program that sums the negative numbers the user entered.
  - Condition - 1: The program will print "Enter a negative number:" and process number till the user enters 0.
  - Condition - 2: If the user enters a positive number, print a warning message then demand to enter a negative number again.
  - The expected screen is shown below:

C:\Users\AKIN\source\repos\IE_Gazi\IE_Gazi\bin\Debug\netcoreapp3.1\IE_Gazi.exe

```
Enter a negative number:6
You have entered a positive number. Try again!
Enter a negative number:-3
Enter a negative number:-4
Enter a negative number:-1
Enter a negative number:0
You have entered 0. Now the program terminates.
Sum of negative number you entered:-8
```

```csharp
// Example for Gazi IE
0 başvuru
static void Main(string[] args)
{
    int num, sum = 0;

    while (true)
    {
        Console.Write("Enter a negative number:");
        num = int.Parse(Console.ReadLine());

        if (num>0)
        {
            Console.WriteLine("You have entered a positive number. Try again!");
            continue;
        }
        if (num==0)
        {
            Console.WriteLine("You have entered 0. Now the program terminates.");
            break;
        }

        sum += num;
    }

    Console.WriteLine("Sum of the negative numbers you entered:{0}", sum);
}
```

THE END

CHECK OUT COURSE WEBSITE: HTTPS://GAZI-END-MUH.GITHUB.IO

GOT ANY QUESTIONS LATER?

SEND ME AN EMAIL: DR.ERCAN.EZIN@GMAIL.COM